

Large Language Models and Alignment

Summer School in AI & Applied Economics

Elliott Ash

ETH Zürich

August 2026

Where we are

- ▶ Yesterday: supervised machine learning — flexible function fitting, with train/test discipline — and embeddings: text and images as vectors whose geometry encodes meaning.
- ▶ This morning: how ML predictions enter econometric designs.
- ▶ Today, in three moves:
 1. how a **language model** works — tokens, attention, the transformer;
 2. what turns a text completer into **ChatGPT** — instruction tuning, RLHF, reasoning;
 3. what economists can **do with the result** — RAG, agents, and a research application.

The recipe

Data (the whole internet) + compute (billions of parameters) + ML (transformer architecture) + **human preference alignment**.

Outline

How a Language Model Works

From GPT to ChatGPT: Instruction Fine-Tuning

RLHF and Preference Learning

Reasoning Models

Retrieval-Augmented Generation (RAG)

AI Agents

Research Application: Proverbs and Cultural Change

Outline

How a Language Model Works

From GPT to ChatGPT: Instruction Fine-Tuning

RLHF and Preference Learning

Reasoning Models

Retrieval-Augmented Generation (RAG)

AI Agents

Research Application: Proverbs and Cultural Change

Tokens: the atoms of a language model

- ▶ A language model does not read words. It reads **tokens**: entries in a fixed vocabulary of roughly 50k–200k character sequences.
- ▶ **Byte-pair encoding** (BPE) builds this vocabulary from the corpus: start from single characters, repeatedly merge the most frequent adjacent pair. Frequent words end up as one token; rare words split into pieces.
- ▶ Consequence: there is no “unknown word” — any string can be encoded — and different forms (“tax”, “taxes”, “taxed”) share subword pieces.

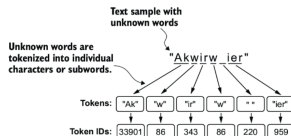


Figure 2.11 BPE tokenizers break down unknown words into subwords and individual characters. This way, a BPE tokenizer can parse any word and doesn't need to replace unknown words with special tokens, such as `<|unk|>`.

Byte-pair encoding splits an unfamiliar word into known subword tokens (Raschka 2024). Try it:
<https://platform.openai.com/tokenizer>

Language modeling = next-token prediction

- ▶ The task: given a corpus, learn $\Pr(w_{i+1} \mid w_{1:i})$ — a probability distribution over the *next* token given everything so far.
- ▶ The classical simplification is the Markov assumption: condition only on the last k tokens, $\Pr(w_{i+1} \mid w_{1:i}) \approx \Pr(w_{i+1} \mid w_{i-k:i})$. Modern LLMs condition on the whole context.
- ▶ The output is not one guess but a **full distribution** over the vocabulary — the log-probabilities behind it are a measurement tool in their own right (Thursday's lecture uses them directly).

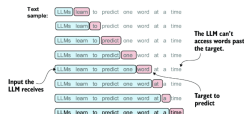


Figure 2.12 Given a text sample, extract input blocks as subsamples that serve as input to the LLM, and the LLM's prediction task during training is to predict the next word that follows the input block. During training, we mask out all words that are past the target. Note that the text shown in this figure must undergo tokenization before the LLM can process it; however, this figure omits the tokenization step for clarity.

Next-token prediction: every position in every sentence of the corpus is a training example (Raschka 2024).

Perplexity: scoring a language model

- ▶ How well does a model predict a corpus of n tokens $\{w_1, \dots, w_n\}$?

$$\text{Perplexity} = 2^{-\frac{1}{n} \sum_{i=1}^n \log_2 \hat{\Pr}(w_i | w_{1:i-1})}$$

- ▶ Lower is better: a good model assigns high probability to the tokens that occur.
- ▶ Interpretation: the effective number of tokens the model is “choosing between” at each step.

Economists use this as data

Perplexity of a statement under a model trained *before* it was written is a **surprise index**: score FOMC statements against a pre-announcement baseline and regress returns on the surprise. (Thursday's lecture shows the code.)

From counting to function fitting

- ▶ The n-gram estimator is just a ratio of corpus counts:

$$\hat{\text{Pr}}(w_{i+1} \mid w_{i-k:i}) = \frac{\#(w_{i-k:i+1})}{\#(w_{i-k:i})}$$

- ▶ It fails for the same reason a saturated regression fails: most cells are empty. Most word sequences never occur, so most estimates are zero or rest on a handful of observations.
- ▶ The **neural** language model replaces counting with smooth function fitting: embed the context tokens as vectors, feed them through a neural network, output a softmax over the vocabulary. Similar contexts now share statistical strength — exactly the embedding logic from yesterday.
- ▶ The network could be an MLP, a CNN, or an RNN. The architecture that won is the **transformer**.

Attention: a learned soft lookup

- ▶ Think of a database of key–value pairs $D = \{(k_1, v_1), \dots, (k_m, v_m)\}$ and a query q . A dictionary lookup returns v_i where $k_i = q$: all-or-nothing weights.
- ▶ **Attention** generalizes the lookup to a weighted average:

$$\text{Attention}(q) = \sum_{i=1}^m a(q, k_i) v_i, \quad a(\cdot) \geq 0, \quad \sum_i a(q, k_i) = 1$$

- ▶ The weights come from a softmax over query–key similarity scores, so they are differentiable — the lookup itself can be *learned* by gradient descent.
- ▶ Linguistic reading: each word gets to query every other word in the document and pull in whatever is predictive.

Self-attention over word embeddings

- ▶ Take a sequence of tokens $\{w_1, \dots, w_{n_L}\}$ with embedding vectors $\{x_1, \dots, x_{n_L}\}$ of dimension n_E .
- ▶ A **self-attention layer** produces a new sequence of **context vectors** $h_1, \dots, h_{n_L}$:

$$h_i = \sum_{j=1}^{n_L} a(x_i, x_j) x_j, \quad a(x_i, x_j) = \text{softmax}\left(\frac{x_i \cdot x_j}{\sqrt{n_E}}\right)$$

- ▶ Each token's query is its own embedding; the keys and values are the other tokens' embeddings. Each h_i is a weighted average of the whole sentence, weighted by similarity to token i .
- ▶ This *basic* form has no parameters of its own — it works entirely through the learned embeddings. The production version (GPT, BERT) adds learned query/key/value projection matrices and multiple parallel **attention heads** per layer.

Why self-attention works

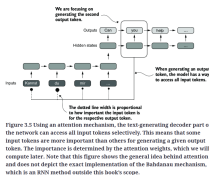
- ▶ Sentence: *the cat walks on the street*, with embeddings

$x_{\text{the}}, x_{\text{cat}}, x_{\text{walks}}, x_{\text{on}}, x_{\text{the}}, x_{\text{street}}$.

- ▶ Each output mixes the whole sentence:

$$h_{\text{cat}} = a(x_{\text{cat}}, x_{\text{the}}) x_{\text{the}} + a(x_{\text{cat}}, x_{\text{cat}}) x_{\text{cat}} + \cdots + a(x_{\text{cat}}, x_{\text{street}}) x_{\text{street}}$$

- ▶ Training shapes the embeddings so the dot products are *useful*: informative pairs (“cat”–“walks”) get weight, uninformative ones (“the”) get ignored.



Attention weights: one token's context vector is a weighted average of every token in the sequence (Raschka 2024).

Position embeddings: putting word order back

- ▶ Self-attention is a weighted average — it is **permutation invariant**. “Dog bites man” and “man bites dog” would encode identically.
- ▶ Fix: add a **position embedding** t_i (a learned vector for position index i) to each word embedding before the first attention layer:

$$h_{1:n_L}^0 = \{x_1 + t_1, \dots, x_{n_L} + t_{n_L}\}$$

- ▶ Modern models instead *rotate* the embeddings by position (RoPE) — an implementation detail for Thursday.

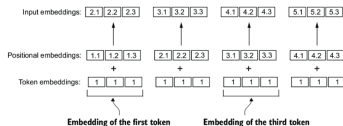
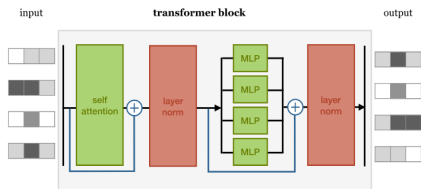


Figure 2.18 Positional embeddings are added to the token embedding vector to create the input embeddings for an LLM. The positional vectors have the same dimension as the original token embeddings. The token embeddings are shown with value 1 for simplicity.

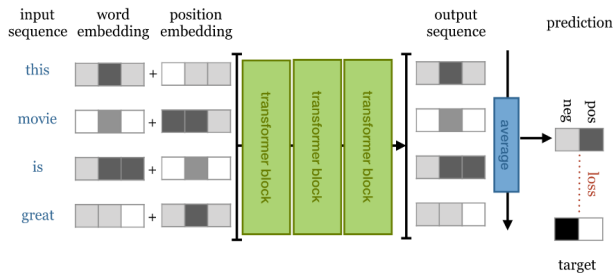
Input to the model = token embedding + position embedding (Raschka 2024).

The transformer block



- ▶ One block = a **self-attention layer** followed by a small **MLP** at each position, plus two tricks that make deep stacks trainable: **residual connections** (the input is added back to each layer's output) and **layer normalization** (each vector rescaled to unit variance).
- ▶ Blocks chain: block l 's output $h_{1:n_L}^l$ is block $l + 1$'s input.
- ▶ The MLPs hold roughly **two thirds** of each block's parameters — remember this Thursday, when mixture-of-experts models multiply exactly that part.

Stack blocks, add a head: a complete model



A transformer for sentiment classification: token + position embeddings, a stack of transformer blocks, pooling to a document vector, and a classification head.

- ▶ Every transformer application is this picture with a different final layer: a classifier head (here), or a softmax over the vocabulary (a language model).
- ▶ This classifier is exactly what Thursday's lecture fine-tunes as the cheap workhorse for labelled text (BERT-style, step 3 of the ladder).

Three kinds of transformer language model

- ▶ **Autoregressive / decoder-only** (GPT): pre-trained to predict the next token, seeing only the past. Built for generation.
- ▶ **Autoencoding / encoder-only** (BERT): pre-trained to fill in a masked token, seeing the whole sequence in both directions. Built for representation — classification, embeddings.
- ▶ **Sequence-to-sequence** (Vaswani et al. 2017; T5, BART): an encoder reads, a decoder writes. Once standard for translation; now niche.

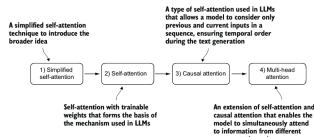


Figure 3.2 The figure depicts different attention mechanisms we will code in this chapter, starting with a simplified version of self-attention before adding the trainable weights. The causal attention mechanism adds a mask to self-attention that allows the LLM to generate one word at a time. Finally, multi-head attention organizes the attention mechanism into multiple heads, allowing the model to capture various aspects of the input data in parallel.

Encoder-only, decoder-only, and encoder–decoder transformers (Raschka 2024).

Causal attention: how a decoder avoids cheating

- ▶ Training predicts every next token at once; if attention could see the future, the model would copy the answer. **Causal (masked) attention** restricts each position to tokens at or before it:

$$\underbrace{h_i = \sum_{j=1}^{n_L} a(x_i, x_j) x_j}_{\text{encoder: sees all}} \longrightarrow \underbrace{h_i = \sum_{j=1}^i a(x_i, x_j) x_j}_{\text{decoder: past only}}$$

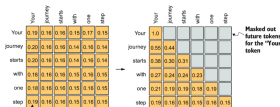
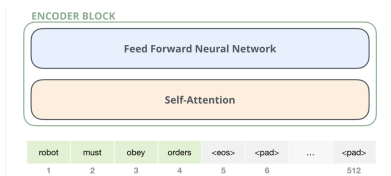


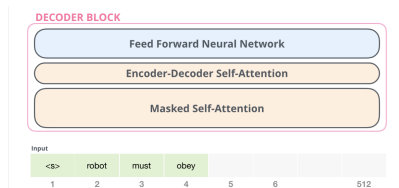
Figure 3.19 In causal attention, we mask out the attention weights above the diagonal such that for a given input, the LLM can't access future tokens when computing the context vectors using the attention weights. For example, for the word "journey" in the second row, we only keep the attention weights for the words before ("Your") and in the current position ("journey").

The causal mask zeroes out attention weights above the diagonal (Raschka 2024).

Encoder reads, decoder writes



An encoder block from the original transformer paper can take inputs up until a certain max sequence length (e.g. 512 tokens). It's okay if an input sequence is shorter than this limit, we can just pad the rest of the sequence.



Encoder block “reads”:

- ▶ observes the whole sequence, both directions;
- ▶ BERT: the classification and embedding workhorse.
- ▶ Interactive anatomy of a decoder:

<https://poloclub.github.io/transformer-explainer/>

Decoder block “writes”:

- ▶ causal attention: never sees future tokens;
- ▶ GPT — and every chat model in this lecture — is decoder-only.

GPT: scale up and pre-train

- ▶ **GPT = Generative Pre-trained Transformer**: a decoder-only stack trained on next-token prediction, then scaled.
 - ▶ **GPT-1** (2018): the Books corpus; 12 blocks, $\approx 117\text{M}$ parameters.
 - ▶ **GPT-2** (2019): 8M web documents (40 GB); 48 blocks, $\approx 1.5\text{B}$ parameters.
 - ▶ **GPT-3** (2020): Common Crawl + books + Wikipedia; 96 blocks, 12,288-dimensional embeddings, $\approx 175\text{B}$ parameters.
- ▶ Nothing conceptually new happens between GPT-1 and GPT-3 — the recipe is *the same objective on more data with more parameters*.
- ▶ What changed is what the models could *do*. That is where the story picks up.

Outline

How a Language Model Works

From GPT to ChatGPT: Instruction Fine-Tuning

RLHF and Preference Learning

Reasoning Models

Retrieval-Augmented Generation (RAG)

AI Agents

Research Application: Proverbs and Cultural Change

The final ingredient

- ▶ By 2020 we had all the raw materials:
 - ▶ **Data:** GPT-2 read 8 billion words; GPT-3 read 114 billion; today's frontier models read on the order of 10 trillion.
 - ▶ **Compute:** GPUs, and models with hundreds of billions of parameters.
 - ▶ **ML:** the transformer architecture.
- ▶ And yet GPT-3 was not a useful product. Something was missing.

Analogy: how do children learn language?

1. They learn to repeat what they hear. ($\approx$ pre-training)
2. They learn explicit rules from parents and teachers. ($\approx$ instruction tuning)
3. They get positive *and negative* feedback when they say something right or wrong. ($\approx$ learning from preferences)

Emergent zero-shot learning (GPT-1, GPT-2)

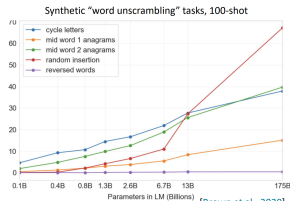
- ▶ Already with GPT (2018), and clearly with GPT-2 (“Language models are unsupervised multitask learners”, 2019), researchers observed **emergent zero-shot learning**.
- ▶ The model was never trained to answer questions or to summarize. But because the pre-training corpus contains such patterns, they can be elicited by formatting the prompt correctly:
 - ▶ Passage: [wikipedia excerpt for X] --- Q: Where was X born?
A: ...
 - ▶ [article] TL;DR: ...
- ▶ **Key idea:** the task specification is smuggled in through the text itself. There is no task-specific head and no labelled training set.

Main source for this section: Jesse Mu’s CS224n slides,

<https://web.stanford.edu/class/cs224n/slides/cs224n-2023-lecture11-prompting-rlhf.pdf>

Emergent few-shot learning (GPT-3)

- ▶ “Few-shot learning”: specify the task and give a handful of worked examples *inside the prompt*, before the query.
- ▶ Became strikingly effective with GPT-3 (“Language models are few-shot learners”, 2020). No gradient updates are involved — this is **in-context learning**.
- ▶ Crucially, the benefit *grows with model scale*.



Accuracy rises with the number of in-context examples, and the gain from in-context examples is much larger for larger models.

Chain-of-thought prompting

- ▶ Asking the model to “think step by step” before answering substantially improves multi-step arithmetic and logical reasoning.
- ▶ The intermediate tokens act as a scratchpad: the model conditions its final answer on its own reasoning trace.
- ▶ This is a purely *prompting* intervention — a preview of the reasoning models we discuss later today.

Chain-of-thought prompting

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

Standard prompting versus chain-of-thought prompting (Wei et al. 2022).

Base language models are not aligned

- ▶ A pre-trained LM maximizes $\Pr(w_{t+1} \mid w_{1:t})$ on internet text. That is *not* the same objective as being helpful.
- ▶ Ask a base model a question and it may continue with more questions, restate the prompt, drift into an unrelated web page, or confidently fabricate.

Language modeling $\neq$ assisting users

PROMPT	<i>Explain the moon landing to a 6 year old in a few sentences.</i>
COMPLETION	GPT-3 Explain the theory of gravity to a 6 year old. Explain the theory of relativity to a 6 year old in a few sentences. Explain the big bang theory to a 6 year old. Explain evolution to a 6 year old.

Language models are not *aligned* with user intent [[Ouyang et al., 2022](#)].

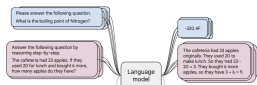
The base model completes text; it does not follow instructions.

Instruction fine-tuning (IFT)

- ▶ **Idea:** continue training the pre-trained model, with the ordinary next-token objective, on a curated set of **(instruction, response)** pairs covering many tasks. Also called **supervised fine-tuning** (SFT).
- ▶ Typically 10k–100k pairs — tiny relative to pre-training, but it reshapes behaviour dramatically.
- ▶ The payoff is **generalization across tasks**: train on a diverse mixture of instructions, and the model then follows *held-out* instruction types it never saw.

Instruction finetuning

- Collect examples of (instruction, output) pairs across many tasks and finetune an LM

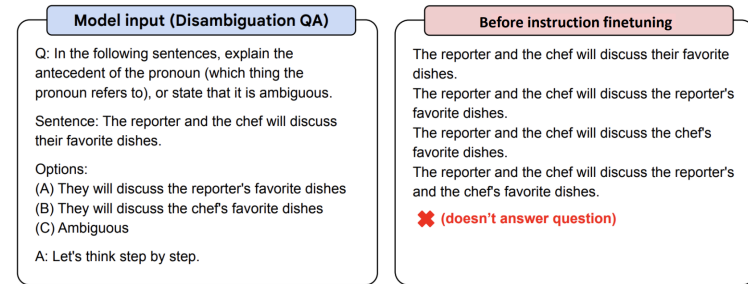


- Evaluate on unseen tasks



Multi-task instruction tuning, as in FLAN-T5.

What the instruction data looks like



Instruction–response pairs used to fine-tune InstructGPT (Ouyang et al. 2022).

- ▶ Prompts come from real API traffic plus labeller-written prompts; responses are written by trained human annotators.
- ▶ Diversity of *task type* matters more than raw volume.

Limitations of instruction fine-tuning

1. **Collecting task data is expensive.** Every example is a human-written demonstration.
2. **Many important tasks cannot be written as instructions with correct answers.** Story generation, advice, summarization at the right level of detail.
3. **Token-level losses penalize all mistakes equally.** A factual error and a stylistic quirk cost the same.
4. **Human-written answers may be suboptimal.** The model is forced to imitate one particular phrasing even when it could produce a better one.

Consequence

We need a training signal defined over *whole outputs* and expressing *relative* quality — not a token-by-token imitation loss.

Outline

How a Language Model Works

From GPT to ChatGPT: Instruction Fine-Tuning

RLHF and Preference Learning

Reasoning Models

Retrieval-Augmented Generation (RAG)

AI Agents

Research Application: Proverbs and Cultural Change

Aligning language models with reinforcement learning

- ▶ Let $p_\theta(y \mid x)$ be a language model with parameters θ , giving the probability of output y given prompt x , with samples $\hat{y}(x)$.
- ▶ Suppose we have a reward (utility) function $U(y, x) \in \mathbb{R}$ — for example a human evaluation of the output. It need *not* be differentiable.
- ▶ We want to choose θ to maximize expected reward over generated samples $\hat{y}_\theta(x) \sim p_\theta(y \mid x)$:

$$\theta^* = \arg \max_{\theta} \mathbb{E}\{U(\hat{y}_\theta(x), x)\}$$

- ▶ By gradient ascent:

$$\theta_{t+1} := \theta_t + \alpha \nabla_{\theta_t} \mathbb{E}\{U(\hat{y}_\theta(x), x)\}$$

- ▶ $\rightarrow$ solvable with **policy gradient** methods from reinforcement learning.

Policy gradients in two slides (1)

- For a sample of generations S ,

$$\nabla_{\theta} \mathbb{E}\{U(\hat{y}_{\theta}(x), x)\} = \sum_{y \in S} U(\hat{y}_{\theta}(x), x) \nabla_{\theta} p_{\theta}(y | x)$$

- The “log derivative trick”:

$$\nabla_{\theta} \log p_{\theta}(y | x) = \frac{1}{p_{\theta}(y | x)} \nabla_{\theta} p_{\theta}(y | x) \implies \nabla_{\theta} p_{\theta}(y | x) = p_{\theta}(y | x) \nabla_{\theta} \log p_{\theta}(y | x)$$

- Substituting:

$$\begin{aligned} \sum_{y \in S} U(\hat{y}_{\theta}(x), x) \nabla_{\theta} p_{\theta}(y | x) &= \sum_{y \in S} p_{\theta}(y | x) U(\hat{y}_{\theta}(x), x) \nabla_{\theta} \log p_{\theta}(y | x) \\ &= \mathbb{E}\{U(\hat{y}_{\theta}(x), x) \nabla_{\theta} \log p_{\theta}(y | x)\} \end{aligned}$$

Policy gradients in two slides (2)

- ▶ The left-hand side cannot be computed directly, because $U(\cdot)$ is not differentiable. The right-hand side can be estimated by Monte Carlo:

$$\nabla_{\theta} \mathbb{E}\{U(\hat{y}_{\theta}(x), x)\} \approx \frac{1}{m} \sum_{i=1}^m U(y_i, x) \nabla_{\theta} \log p_{\theta}(y_i | x)$$

where the y_i are sampled from the model itself.

- ▶ So the gradient ascent update is

$$\theta_{t+1} := \theta_t + \alpha \frac{1}{m} \sum_{i=1}^m U(y_i, x) \nabla_{\theta} \log p_{\theta}(y_i | x)$$

- ▶ if $U(y_i, x)$ is positive, change θ to *increase* $p_{\theta}(y_i | x)$;
 - ▶ if $U(y_i, x)$ is negative, change θ to *decrease* it.
- ▶ Under regularity conditions this converges to θ^* .

How to model human preferences $U(\cdot)$

Problem 1: a human in the loop is far too expensive to evaluate $U(y, x)$ for every y, x encountered during training.

- ▶ Solution: train a **reward model** $U_\phi(y, x)$ — itself a fine-tuned LM with a scalar regression head — and use it instead of ground truth.

Problem 2: human preference scores are noisy and poorly calibrated across annotators.

- ▶ Solution: use **pairwise comparisons** — revealed preference! — rather than cardinal scores. Fit a Bradley–Terry model by minimizing

$$L(\phi) = -\frac{1}{|D|} \sum_D \log \sigma(U_\phi(y_0, x) - U_\phi(y_1, x))$$

where D is a dataset of pairs (y_0, y_1) and y_0 is the preferred output.

Interpretation for economists

U_ϕ is an estimated utility function of the representative annotator.

InstructGPT: where the preference data comes from

- ▶ Annotation team for InstructGPT: about **40 contractors** recruited through Upwork and ScaleAI.
- ▶ Screened with a qualification task covering flagging sensitive content, judging completion quality, writing demonstrations, and sensitivity to demographic group membership.
- ▶ Annotators see one prompt with several candidate completions, and rank them.

The cost of preferences

Nathan Lambert: “From the Llama 2 numbers and other rumors, it’s likely Meta spent \$10–20 million plus on preference data. This is just for the final model and may not include extensive experimentation and evaluation.”

- ▶ **Who these annotators are is an economic and political question**, not just an engineering detail.

RLHF: the full objective

Ingredients:

- ▶ a pre-trained (and instruction-tuned) model $p_{\theta}^{PT}(y | x)$, with θ now frozen;
- ▶ a reward model $U_{\phi}(y, x)$ estimated from pairwise comparisons.

Procedure:

- ▶ Make a trainable copy p_{ψ}^{RL} , initialized at p_{θ}^{PT} .
- ▶ Use RL on samples $S \sim p_{\psi}^{RL}$ to solve

$$\psi^* = \arg \max_{\psi} \sum_{y \in S} \left(U_{\phi}(y, x) - \beta \log \frac{p_{\psi}^{RL}(y | x)}{p_{\theta}^{PT}(y | x)} \right)$$

- ▶ The second term is a **KL penalty** for diverging from the pre-trained model. Without it the policy collapses onto degenerate text that games the reward model.
- ▶ β is a hyperparameter (e.g. $\beta = 0.05$ in Stiennon et al. 2020).

Does it work? SFT + RLHF > SFT

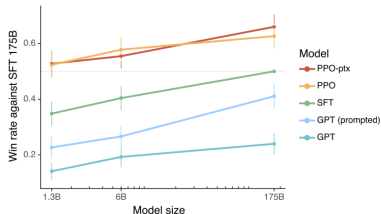


Figure 1: Human evaluations of various models on our API prompt distribution, evaluated by how often outputs from each model were preferred to those from the 175B SFT model. **Our InstructGPT models (PPO-ptx) as well as its variant trained without pretraining mix (PPO) significantly outperform the GPT-3 baselines (GPT, GPT prompted);** outputs from our 1.3B PPO-ptx model are preferred to those from the 175B GPT-3. Error bars throughout the paper are 95% confidence intervals.

Human preference win rates against a reference model, by model size and training stage (Ouyang et al. 2022). A 1.3B-parameter RLHF model is preferred to the 175B-parameter base GPT-3.

- Alignment buys more than two orders of magnitude of scale on this metric.

Why RLHF, and not just more supervised data? (Goldberg 2023)

1. **Flexibility of wording.** Under imitation learning the model must reproduce the annotator's exact phrasing. But many wordings convey the same correct answer; SFT punishes the model for perfectly good alternatives it already knows how to produce.
2. **Positive and negative feedback.** SFT provides only positive examples; RLHF can say “not that”. In formal learning theory, without negative feedback an adversarial teacher can mislead the learner by withholding examples. *GPT-3 knows what to say; ChatGPT knows what **not** to say.*
3. **Preventing hallucination.** For knowledge-seeking queries the model must be able to *refuse*. SFT effectively teaches it to lie: since every demonstration contains an answer, the model learns always to produce one. RLHF can reward abstention.

Beyond PPO: DPO and GRPO

- ▶ **PPO** (proximal policy optimization) is on-policy: you must hold the policy, a frozen reference model, the reward model, and a value model in memory at once. Slow and fiddly.
- ▶ **DPO** (direct preference optimization): the KL-regularized objective has a closed-form optimal policy. Invert it and optimize preferences *directly* with a classification loss — no reward model, no sampling loop.
- ▶ **GRPO** (used in DeepSeek-R1): drop the learned value model. Sample a *group* of completions per prompt and use the group's mean reward as the baseline. Cheaper and more stable.

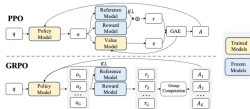


Figure 4 | Demonstration of PPO and our GRPO. GRPO foregoes the value model, instead estimating the baseline from group scores, significantly reducing training resources.

PPO versus GRPO: GRPO replaces the learned value function with a group-relative advantage baseline.

How smart can RLHF models get?

Andrej Karpathy:

- ▶ *“Instead of the mysticism of ‘asking an AI’, think of it more as ‘asking the average data labeler’ on the internet.”* With the caveat that in code, math, and creative writing the companies hire skilled labellers.
- ▶ *“RLHF moves the performance to ‘discriminative human’ grade, up from SFT which is at ‘generative human’ grade . . . because discrimination is easier for an average person than generation”* — label which of these five poems is best versus write a poem.
- ▶ *“You also get a separate boost from the wisdom of crowds effect . . . your LLM performance is not at human level, but at ensemble-of-human level.”*

The implied ceiling

With RLHF alone, the best you can hope for is a model whose answers a panel of top human experts would prefer over any alternative. **To go beyond human preferences you need a different reward signal** — which is where we go next.

Outline

How a Language Model Works

From GPT to ChatGPT: Instruction Fine-Tuning

RLHF and Preference Learning

Reasoning Models

Retrieval-Augmented Generation (RAG)

AI Agents

Research Application: Proverbs and Cultural Change

What was still missing after GPT-4?

The “Sparks of AGI” paper (April 2023) catalogued failures that scale alone did not fix: arithmetic slips, planning errors, inability to backtrack, confident wrong answers.

- ▶ At least two things were missing:
 1. **information retrieval**, to access a fresh knowledge base;
 2. the ability to **take actions** and interact with an environment, rather than only writing text.
- ▶ And arguably a third:
 3. the ability to **reliably extrapolate and reason** through exploration and self-correction.

Item 3 is this section; items 1 and 2 are the next two sections.

How LLMs got good at math

Category

Math

Apply filter

☐ Style Control

☐ Show Deprecated

#models: 227 (99%) #votes: 384,871 (13%)

Rank* (UB)	Delta	Model	Arena Score	95% CI	Votes	Organization	License
1	0	Gemini-2.5-Pro-Exp-03-25	1417	+17/-20	1152	Google	Proprietary
1	1	o3-2025-04-16	1403	+39/-42	241	OpenAI	Proprietary
1	8	o4-mini-2025-04-16	1400	+43/-36	200	OpenAI	Proprietary
2	2	GPT-4.5-Preview	1377	+12/-17	1626	OpenAI	Proprietary
2	1	Gemini-2.5-Flash-Preview-04-17	1363	+20/-22	453	Google	Proprietary
2	7	DeepSeek-R1	1361	+13/-14	1601	DeepSeek	MIT
2	15	o3-mini-high	1359	+12/-13	2032	OpenAI	Proprietary
3	0	o1-2024-12-17	1354	+9/-9	3252	OpenAI	Proprietary

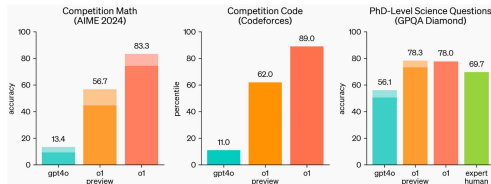
Math performance on the LMSYS leaderboard. A discrete jump appears with o1-class models, Gemini 2.5, and DeepSeek-R1.

What is different about these models?

Scaling test-time compute

- ▶ Classic scaling: spend more compute at *training* time.
- ▶ New axis: spend more compute at *inference* time — let the model generate a long internal reasoning trace before answering.
- ▶ Accuracy improves roughly log-linearly in the number of thinking tokens, over several orders of magnitude.

Highlight of LLMs in 2024: the advancement of reasoning models



OpenAI o1 started to achieve impressive performance across various challenging reasoning tasks in math, coding, STEM, etc.

Inference-time techniques for reasoning. Source: Berkeley RDI, Advanced LLM Agents course.

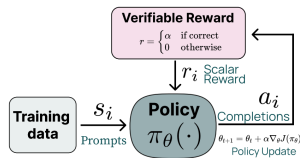
What “reasoning” models add

Relative to a standard RLHF chat model, a model like DeepSeek-R1 adds three ingredients:

1. **Long internal traces.** The model is allowed to “think” and “reflect”, generating many internally observed tokens before emitting an external answer.
2. **Reasoning SFT.** Additional supervised fine-tuning on a large set of worked examples containing step-by-step derivations, verifications, and self-corrections.
3. **RLVR — reinforcement learning from verifiable rewards.** The model solves math and code problems by guessing and verifying, and receives reward only when the answer is provably correct.

<https://www.interconnects.ai/p/deepseek-r1-recipe-for-o1>

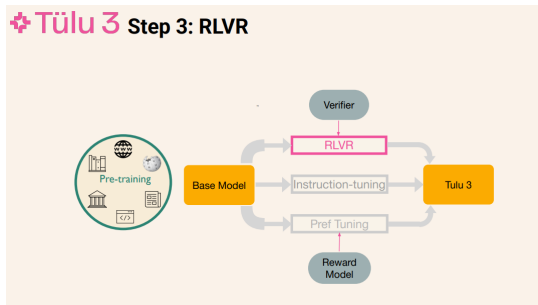
RLVR: reinforcement learning from verifiable rewards



A verifier replaces the learned reward model.

- ▶ Confront the model with thousands of math and logic problems; let it guess, reflect, and verify until it finds a checkable solution. Reward only if correct.
- ▶ **Why this matters:** the reward is no longer a noisy estimate of average human preference. It is ground truth — so the human-preference ceiling does not bind. It also reduces hallucination, by teaching self-critique.
- ▶ **Limitation:** it needs a verifier. Easy for math, code, and formal logic; hard for most social-science questions.

RLVR in an open post-training pipeline



Where RLVR sits in the Tülu-style post-training recipe: SFT, then preference tuning, then RL with verifiable rewards.

- Post-training is now a *pipeline*, not a single step, and each stage is separately measurable.



2.2.2. Reward Modeling

The reward is the source of the training signal, which decides the optimization direction of RL. To train DeepSeek-R1-Zero, we adopt a rule-based reward system that mainly consists of two types of rewards:

- **Accuracy rewards:** The accuracy reward model evaluates whether the response is correct. For example, in the case of math problems with deterministic results, the model is required to provide the final answer in a specified format (e.g., within a box), enabling reliable rule-based verification of correctness. Similarly, for LeetCode problems, a compiler can be used to generate feedback based on predefined test cases.
- **Format rewards:** In addition to the accuracy reward model, we employ a format reward model that enforces the model to put its thinking process between '`<think>`' and '`</think>`' tags.

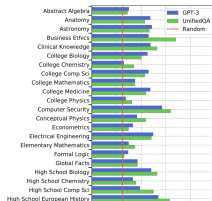
DeepSeek-R1 benchmark performance against closed frontier models.

- ▶ During post-training, DeepSeek is aligned on *both* the RLHF and the RLVR objective: helpful and honest, and also correct.
- ▶ Released with open weights and a detailed recipe — which is why it matters for academic research budgets.

Evaluating LLMs: knowledge benchmarks

**Massive Multitask Language
Understanding (MMLU)**
[\[Hendrycks et al., 2021\]](#)

New benchmarks for measuring LM
performance on 57 diverse *knowledge
intensive* tasks



MMLU: 57 knowledge-intensive multiple-choice tasks (Hendrycks et al. 2021).

- ▶ **MMLU**: about 16k multiple-choice questions across 57 subjects.
- ▶ **BIG-Bench**: a large heterogeneous suite scored with exact-match and string-similarity metrics.
- ▶ **GPQA**: 26,529 graduate-level, “Google-proof” questions across 285 fields.
- ▶ **Caution**: benchmarks leak into training corpora. Contamination makes published numbers hard to interpret.

Evaluating LLMs: LLM-as-judge

Arena-Hard-Auto (<https://arxiv.org/abs/2406.11939>):

- ▶ **Tasks:** 500 challenging real user queries (open-ended software engineering problems, math questions) plus 250 creative-writing queries sourced from Chatbot Arena.
- ▶ **Evaluation:** frontier LLMs act as judges, making *pairwise* comparisons between two models' answers — a cheap, fast approximation to human preference. The paper reports 98.6% correlation with human judgments.
- ▶ The judge writes its own answer first, then identifies errors in both candidates, then emits a verdict such as `[[A>B]]`.

Relevance for research design

LLM-as-judge is now a standard measurement instrument in applied work. Treat it like any other coder: report agreement with human labels on a validation sample, and worry about systematic bias (position effects, verbosity bias, self-preference).

Outline

How a Language Model Works

From GPT to ChatGPT: Instruction Fine-Tuning

RLHF and Preference Learning

Reasoning Models

Retrieval-Augmented Generation (RAG)

AI Agents

Research Application: Proverbs and Cultural Change

“Hallucination” in LLMs

- ▶ Ask a 2022-vintage ChatGPT “who is the US president today” and you will not get a good answer.
- ▶ Even with a 2025 model, niche questions — “who is the mayor of a small town in Graubünden?” — often produce confident, wrong answers.
- ▶ These wrong answers are called **hallucinations**: fluent, plausible, false.
- ▶ **Why they happen**: the fact was absent from pre-training, or appeared too rarely and too indirectly for the model to recall reliably from its internal world representation. The model has no mechanism for noticing the gap.

For empirical work

Hallucination is a *measurement error* problem, and it is not classical: it is correlated with obscurity, which is often correlated with the variables we care about — small places, minority groups, older documents.

Hallucination is systematic, not incidental

How ~~children~~ ChatGPT learns language

1. **word prediction** – they learn to repeat what they hear.
2. **instruction fine tuning** – they learn some explicit rules.
3. **learning from feedback** – they get positive and negative feedback when they say something right or wrong.

- GPT-1/2/3 (until 2020) only have step 1.
- ChatGPT+ (starting 2022) have steps 2 and 3 (alignment).

Hallucination rates persist across model generations.

- ▶ RLHF helps at the margin — the model learns that it *may* refuse — but does not solve the problem: the training signal still rewards a plausible answer over an admission of ignorance.

Two solutions that do not work

1. **Re-train the model every day.** Infeasible on cost, and it would not help with facts that are rare rather than recent.
2. **Put all the facts in the system prompt.** Context windows are large but not that large, and cost scales with context length.

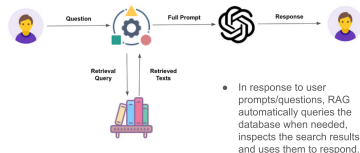
The workable answer

Give the model *selective* access to an external, trustworthy knowledge base at query time — **retrieval-augmented generation**.

Addressing hallucination with RAG

- ▶ Retrieval-augmented generation gives the model access to:
 - ▶ a database of useful and reliable knowledge, and
 - ▶ a search engine for retrieving from it efficiently.
- ▶ Retrieved passages are inserted into the prompt, and the model answers *conditional on the evidence*.
- ▶ Bonus: RAG returns **citations** to source documents, so a human — or a referee — can check the claim.

RAG = Retrieval Augmented Generation



The RAG pipeline: embed the query, search the index, retrieve the top-*k* chunks, and generate conditional on them.

Two canonical RAG tasks

Open question answering

- ▶ Answer an arbitrary question: “What are the responsibilities of the mayor of Zurich?”

Open claim verification

- ▶ Decide whether a plain-text claim is true or false: “Zurich has the second-highest per-capita income of any city in Europe.”

Both are solved with the same information-retrieval pipeline:

1. search large corpora or knowledge graphs for evidence;
2. use the evidence to answer the question or adjudicate the claim.

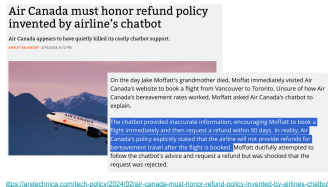
Research uses

Fact-checking political speech; linking firms to filings; grounding a coding scheme in a statute or contract; auditing what a model asserts about a document.

How to index the documents

- ▶ **Sparse / lexical: BM25.** Rank documents by a TF-IDF-like similarity to the query, using an inverted index over words to discard irrelevant documents fast. Very fast, but requires exact word overlap.
- ▶ **Dense / embedding-based.** Encode chunks and queries with a sentence-embedding model (e.g. S-BERT from SentenceTransformers), and retrieve by cosine similarity.
 - ▶ Handles paraphrase and synonymy.
 - ▶ Documents must be segmented into **overlapping chunks** to fit the encoder's context.
 - ▶ Approximate nearest-neighbour search (e.g. FAISS, Johnson et al. 2017) makes this fast at the scale of tens of millions of chunks.
- ▶ **Hybrid** is standard practice: BM25 for a cheap first pass, then embeddings or a cross-encoder to re-rank the top results.

RAG is an active research area — and it fails



An airline was held liable for its chatbot's incorrect statement of its own refund policy.

- ▶ Open directions: use an LLM to remove false positives and duplicates; use an LLM to rewrite or diversify the query; embeddings that exploit metadata and citation networks; tools from recommender systems.
- ▶ **Inference is easier than retrieval.** Modern LLMs are so good at reading a provided document that almost all open-QA errors come from the *retriever*. Once the correct evidence is in context, the answer follows. → high returns to “context engineering”.

The context window revolution

- ▶ Context windows have exploded:
 - ▶ GPT-3.5 (2022): 4K tokens (≈ 6 pages)
 - ▶ GPT-4 (2023): 8K tokens (≈ 12 pages)
 - ▶ Claude Sonnet 4 (2025): 200K tokens (≈ 700 pages)
 - ▶ Gemini 2.5 (2025): 1M tokens ($\approx 3,000$ pages)
- ▶ When context is abundant, **search becomes navigation**. Instead of retrieving fragments, load full documents, follow references, and build understanding.

Practical implication

For a corpus of a few hundred documents — a typical applied-micro setting — you may not need a retrieval pipeline at all. Build one only when the corpus genuinely exceeds what you can afford to put in context.

Agentic search vs. RAG (Bustamante 2025)

- ▶ Traditional RAG is a long pipeline: chunk → embed → retrieve → rerank → generate. Each step can fail, and errors cascade.
- ▶ **Agentic search:** give the model simple tools (grep, file navigation, a shell) and let it investigate iteratively — reading full documents, following cross-references, and accumulating understanding.

Illustration: “What is the company’s litigation exposure?”

- ▶ RAG returns chunks mentioning “litigation” → finds \$500M.
- ▶ An agent follows cross-references across sections and footnotes → finds \$5.1B, ten times more.

“The future belongs to agents that navigate and reason, not pipelines that retrieve fragments.”

<https://www.nicolasbustamante.com/p/the-rag-obituary-killed-by-agents>

Outline

How a Language Model Works

From GPT to ChatGPT: Instruction Fine-Tuning

RLHF and Preference Learning

Reasoning Models

Retrieval-Augmented Generation (RAG)

AI Agents

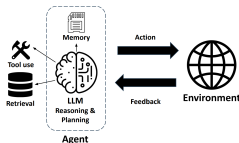
Research Application: Proverbs and Cultural Change

AI agents

Agent systems (Codex, Claude Code, OpenCode) give a language model:

- ▶ the ability and the motivation to **reason and plan**;
- ▶ access to an **external environment** — the internet, or your file system — where it can explore, perceive, and act;
- ▶ access to **tools** (APIs, a shell, a Python interpreter), and **working memory** for organizing actions and plans;
- ▶ the disposition to **take actions** in pursuit of a plan.

LLM agents: enabling LLMs to interact with the environment



The agent loop. Source: Berkeley RDI, LLM Agents course.

MCP: APIs for LLMs

- ▶ **MCP** (Model Context Protocol) standardizes how models interact with external tools — effectively an API convention written for models rather than for developers.
- ▶ Key difference:
 - ▶ ordinary API: the *developer* decides when a function is called;
 - ▶ MCP: the *model* decides when and how to call the tool.
- ▶ What it enables: access to data (databases, files, APIs), execution of actions, and multi-step reasoning workflows.

Analogy

APIs say “call this function.” MCP says “here are tools — use them to achieve a goal.”

Outline

How a Language Model Works

From GPT to ChatGPT: Instruction Fine-Tuning

RLHF and Preference Learning

Reasoning Models

Retrieval-Augmented Generation (RAG)

AI Agents

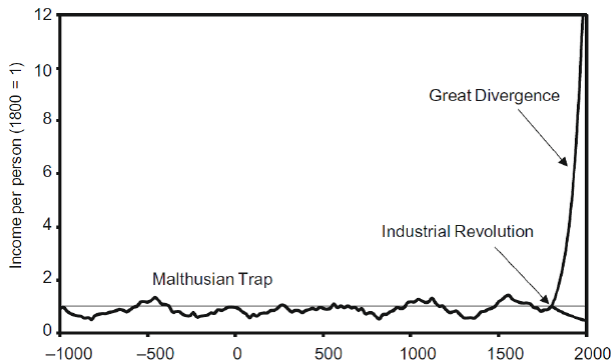
Research Application: Proverbs and Cultural Change

British Industrialization and Cultural Change: Evidence from the Usage of Proverbs

Elliott Ash, ETH Zurich and CEPR
Melanie Meng Xue, LSE and CEPR

December 2025

The Industrial Revolution



Income per person in the UK (Clark 2014, p. 219).

- “The most fundamental transformation of human life in the history of the world recorded in written documents.” (Hobsbawm, 1969)

Proverbs and culture

**“Hold on, wait for the grasshoppers.” “Many get into a dispute
well that cannot get out well.”**

- ▶ Proverbs tend to be metaphorical and cryptic, with unusual syntactic structure:
 - ▶ that makes them easy to memorize and to transmit across generations, resistant to information loss or manipulation (Villers, 2022);
 - ▶ it also makes them hard to understand just from reading them — you need external knowledge.

New data on historical British proverbs

- ▶ **Lists of proverbs:** 26,000 collected from 5 historical sources — OCR'd the PDFs, regex extraction, post-processing, checking, cleaning.
- ▶ **Pre-industrial usage:** EEBO (Early English Books Online) — 17 million pages from 146,000 books, 1475–1750.
- ▶ **Post-industrial usage:** British Newspaper Archive — 40 million pages from more than 1,000 newspaper outlets, 1750–1900.

Explaining proverbs with AI

- ▶ Key challenge: we need not just the proverbs but **explanations** that consistently lay out their social implications.
 - ▶ Would require thousands of hours of work by human expert annotators.
 - ▶ Standard NLP tools do not work with metaphor or implicit meaning.

What LLMs can do that makes this project possible

- ▶ draw on a giant multilingual text knowledge base;
- ▶ interpret metaphorical language;
- ▶ infer and interpolate meaning.

What can we do now that we could not do before?

- ▶ **20 years ago:** annotate proverb explanations along some dimension, machine-classify the rest; learn LDA topics on the explanations.
 - ▶ **But how do we get the explanations?**
- ▶ **7 years ago:** use BERT for more accurate classification from labels; BERTopic for better, context-sensitive topics than LDA.
 - ▶ **But again: how do we get the explanations?**
- ▶ **3 years ago (RLHF models): use an aligned LLM to translate each proverb into an explanation of its cultural meaning** — then apply the familiar tools (BERT, topic models) to classify and interpret the explanations.

What about the future?

- ▶ **Today (RLHF LLMs):** LLM translates proverb → explanation; classifiers and topic models interpret the explanations.
- ▶ **Also today, but experimental (RLVR LLM agents):**
 - ▶ “search the web and give me a list of all English-language proverbs emphasizing the value of patience”;
 - ▶ “count the usage of these proverbs in a provided corpus by county and over time”.
- ▶ **What LLMs cannot do, and won’t be able to do:**
 - ▶ “count the usage of these proverbs by county and over time, but the corpus is not online”;
 - ▶ “give me a measure of the counties in England over time by their cultural valuing of patience”;
 - ▶ “what is the causal effect of industrialization on the valuing of patience?”

The prompt (GPT-4o)

You are a helpful research assistant with knowledge of the literary history of proverbs. Explain the meaning of this proverb in a few sentences of plain language. Interpret the proverb in terms of the implied beliefs, values, attitudes, norms, or ideology.

Structure your answer as follows: "This proverb means that... If a person uses this proverb, it implies that this person ... If this proverb is prevalent in a society, it implies that the society...".

Prompting matters (1): two versions of the ask

Old prompt:

Explain the meaning of this proverb in a few sentences of plain language, including an assessment of its moral, behavioral, or ethical implications. Begin your answer with: "This proverb means that...".

New prompt:

Explain the meaning of this proverb in a few sentences of plain language. Interpret the proverb in terms of the implied beliefs, values, attitudes, norms, or ideology. Structure your answer as follows: "This proverb means that... If a person uses this proverb, it implies that this person ... If this proverb is prevalent in a society, it implies that the society...".

Prompting matters (2): what changes in the output

Proverb: What belongs to the master is forbidden to the slave.

Old explanation: This proverb means that there are certain privileges or possessions that are reserved for those in positions of power . . . **The moral or ethical implication of this proverb is a critique of social injustice and the unfair distribution of resources and rights.**

New explanation: This proverb means that what is owned or controlled by someone in a position of power is not accessible to those in a subordinate position. If a person uses this proverb, it implies that this person recognizes and accepts the strict boundaries and inequalities between social classes. **If this proverb is prevalent in a society, it implies that the society has a clear hierarchy and reinforces the idea that those in power have exclusive rights over their possessions and privileges.**

- ▶ The old prompt made the LLM *moralize*; the new prompt makes it *describe* — the difference between an editorial and a measurement.

How to write prompts

1. Use your best knowledge and intuition.
2. LLMs are very good at following long, detailed instructions, as long as they are coherent. Be specific.
3. Ask an LLM for advice on improving a prompt.
4. Try the prompt on a small sample; find errors and edge cases; update.
5. Prepare a longer human validation set; find further edge cases.
6. Use Co-DETECT (<https://github.com/EdisonNi-hku/Co-DETECT/>) to systematically find and address edge cases (Xiong et al. 2025).

Clustering the explanations with BERTopic works well

A. Patience



Hold on, wait for the grasshoppers.

Haste comes late in the end.

He who pours water hastily into a bottle spills more than goes in.

The Roman conquered by delay.

Haste makes waste and waste makes want, and want makes strife between the good man and his wife.

Nothing is well ordered that is hasty or precipitate.

He tires betimes that spurs too fast betimes, with eager feeding food will choke the feeder.

B. Managing Conflict



Many get into a dispute well that cannot get out well.

It is better to keep out of a quarrel than to make it up afterwards.

Dissensions like small streams at first begun, scarce seen they rise but gather as they run.

It is madness for a sheep to treat of peace with a wolf.

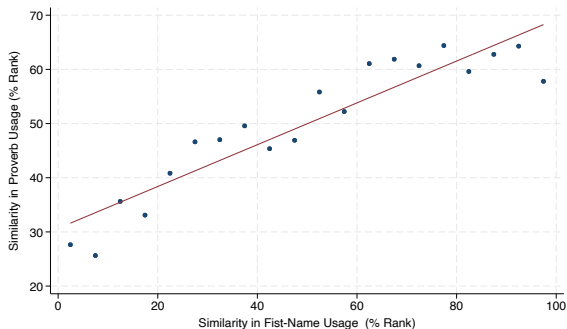
Wranglers never want words though they may matter.

Take things always by the smooth handle.

Happy is the man who keeps out of strife.

Topic clusters over proverb explanations. (LLMs can't really learn topics inductively — clustering still does that job.)

Proverb usage captures cultural distance



County-to-county cosine similarity in proverb topic frequencies, 1800–1900, against cosine similarity in first-name frequencies from the 1851 census. Spearman correlation = 0.26.

LLM Prompt: Work Ethic (Thompson 1967)

You are a helpful research assistant with knowledge of the literary and social history of proverbs. I will provide you with a proverb and its meaning.

Does the proverb emphasize the importance of work ethic, that is, a belief that work and diligence have a moral benefit and an inherent ability, virtue or value to strengthen character and individual abilities?

Read the proverb and its associated meaning and classify it into one of the following categories:

- "Definitely Yes": clearly, directly and strongly emphasizes the importance of work ethic.
- "Probably Yes": suggests the importance of work ethic, though less directly and strongly than "Definitely Yes".
- "Probably Not": suggests that work ethic is not important, though less directly and strongly than "Definitely Not".
- "Definitely Not": clearly, directly and strongly contradicts or dismisses the importance of work ethic.
- "Unrelated": unrelated to work ethic.

Make sure that do not use any exterior or prior knowledge you have when analyzing and do not infer meaning beyond what is explicitly stated. Base your answers only on the proverb provided.

Please also provide a short one-sentence explanation for your classification.

- **Work Ethic (Thompson 1967)**

- ① First deserve, and then desire.
- ② Do not go to sleep early and rise late.
- ③ As is the garden such is the gardener.
- ④ Things hardly attained are longer retained .

Validations

- ▶ Human comparisons of our explanations to dictionaries.
- ▶ Human validation of the annotations of cultural dimensions.
- ▶ Validation that explanations and annotations make sense in context when used in newspapers, books, and web sites.
- ▶ Comparisons across other LLMs and other prompts.
- ▶ Intruder detection on the clusters.

The measurement lesson

An LLM is a coder, not an oracle: validate its output the way you would validate any human annotator.

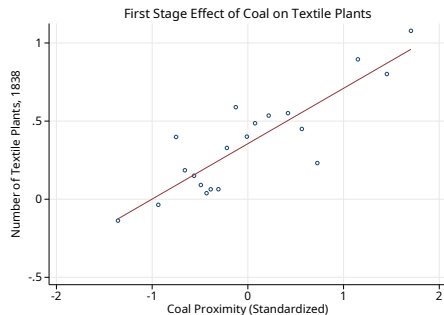
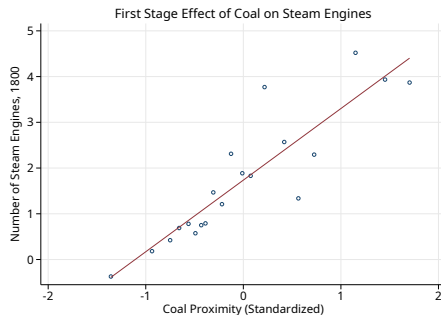
Cultural change in the Industrial Revolution

- ▶ Industrialization transformed England economically and culturally from the late 1700s, with labor shifting out of agriculture into manufacturing and services (Broadberry et al., 2015; Wallis et al., 2018).
- ▶ Thompson (1964): anecdotal evidence on the rise of a “working class” identity, and especially changes in attitudes around *time*.

The design

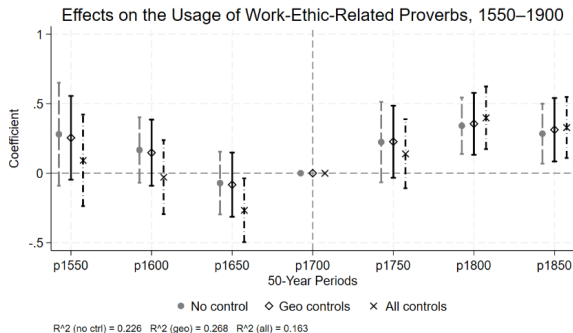
Assess how local proverb usage changed in localities experiencing industrialization, using **presence of coal reserves** as an exogenous shifter to earlier/more industrialization.

First-stage regression results



Steam engines in 1800 (left) and textile plants in 1838 (right) by county, against the share of parishes with coal. Both significant at $p < .01$; Kleibergen–Paap F-stat > 50 .

EFFECT OF INDUSTRIALIZING ON VALUING OF WORK ETHIC



Notes. Regression of usage of proverbs valuing work ethic by 50-year period on coal proximity for that county, interacted with period indicators. Regression include county and period FE's. 90% confidence intervals constructed with standard errors clustered by county.

Summary

1. A **language model** is next-token prediction; the **transformer** makes it work at scale, by letting every token attend to every other.
2. **Instruction fine-tuning** turns a text completer into an assistant cheaply, but it can only imitate.
3. **RLHF** learns a utility function from revealed pairwise preferences and maximizes it under a KL constraint. Its ceiling is expert-panel human preference.
4. **Reasoning models** break that ceiling by replacing preference rewards with *verifiable* rewards, and by spending compute at inference time.
5. **RAG** addresses hallucination by grounding generation in retrieved evidence — and is increasingly giving way to agentic navigation of long contexts.
6. LLMs unlock designs that were impossible before: in the proverbs project, an aligned LLM writes consistent explanations of 26,000 metaphorical proverbs — thousands of hours of expert annotation — turning them into a measure of cultural change. Validate the LLM like any other coder.